MODULE 2: INPUT/OUTPUT ORGANIZATION

ACCESSING I/O-DEVICES

- A **single bus-structure** can be used for connecting I/O-devices to a computer (Figure 7.1).
- Each I/O device is assigned a unique set of address.
- Bus consists of 3 sets of lines to carry address, data & control signals.
- When processor places an address on address-lines, the intended-device responds to the command.
- The processor requests either a read or write-operation.
- The requested-data are transferred over the data-lines.

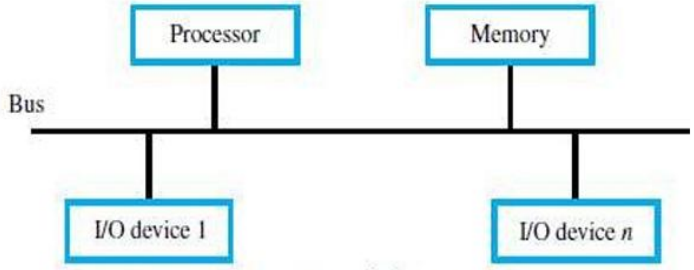


Figure 7.1 A single-bus structure.

- There are 2 ways to deal with I/O-devices: 1) Memory-mapped I/O & 2) I/O-mapped I/O.

1) Memory-Mapped I/O

- Memory and I/O-devices share a common address-space.
- Any data-transfer instruction (like Move, Load) can be used to exchange information.
- For example,

Move DATAIN, R0; This instruction sends the contents of location DATAIN to register R0.
Here, DATAIN → address of the input-buffer of the keyboard.

2) I/O-Mapped I/O

- Memory and I/0 address-spaces are different.
- A special instructions named **IN** and **OUT** are used for data-transfer.
- Advantage of separate I/O space: I/O-devices deal with fewer address-lines.

I/O Interface for an Input Device

1) Address Decoder: enables the device to recognize its address when this address appears on the address-lines (Figure 7.2).
2) Status Register: contains information relevant to operation of I/O-device.
3) Data Register: holds data being transferred to or from processor. There are 2 types:
i) DATAIN → Input-buffer associated with keyboard.
ii) DATAOUT → Output data buffer of a display/printer.

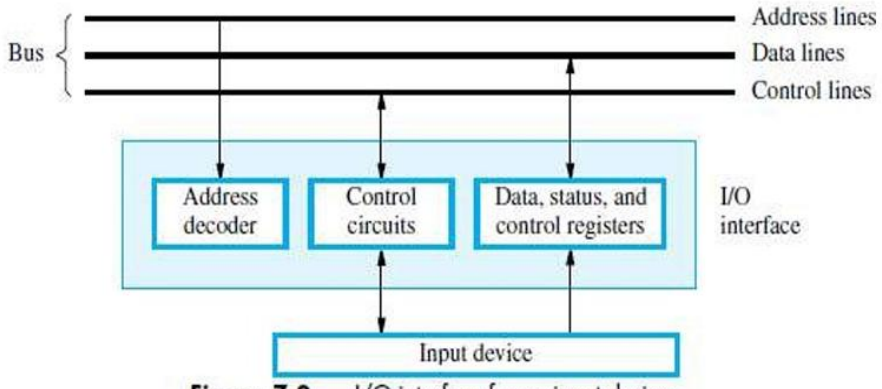


Figure 7.2 I/O interface for an input device.

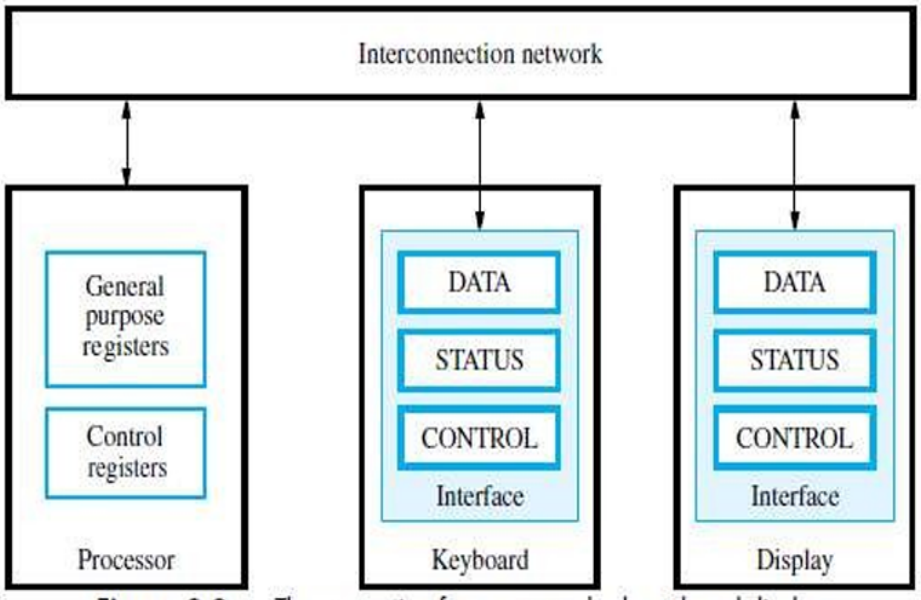


Figure 3.2 The connection for processor, keyboard, and display.

MECHANISMS USED FOR INTERFACING I/O-DEVICES

1) Program Controlled I/O

• Processor repeatedly checks status-flag to achieve required synchronization b/w processor & I/O device. (We say that the processor polls the device).

• Main drawback:

The processor wastes time in checking status of device before actual data-transfer takes place.

2) Interrupt I/O

• I/O-device initiates the action instead of the processor.

• I/O-device sends an INTR signal over bus whenever it is ready for a data-transfer operation.

• Like this, required synchronization is done between processor & I/O device.

3) Direct Memory Access (DMA)

• Device-interface transfer data directly to/from the memory w/o continuous involvement by the processor.

• DMA is a technique used for high speed I/O-device.

INTERRUPTS

- There are many situations where other tasks can be performed while waiting for an I/O device to become ready.
- A hardware signal called an Interrupt will alert the processor when an I/O device becomes ready.
- Interrupt-signal is sent on the interrupt-request line.
- The processor can be performing its own task without the need to continuously check the I/O-device.
- The routine executed in response to an interrupt-request is called ISR.
- The processor must inform the device that its request has been recognized by sending INTA signal.
  (INTR → Interrupt Request, INTA → Interrupt Acknowledge, ISR → Interrupt Service Routine)
- For example, consider COMPUTE and PRINT routines (Figure 3.6).

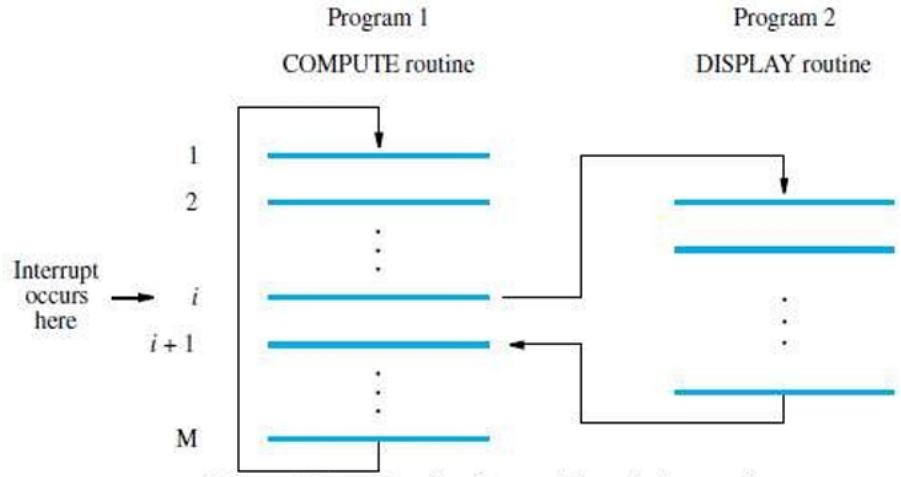


Figure 3.6 Transfer of control through the use of interrupts.

- The processor first completes the execution of instruction i.
- Then, processor loads the PC with the address of the first instruction of the ISR.
- After the execution of ISR, the processor has to come back to instruction i+1.
- Therefore, when an interrupt occurs, the current content of PC is put in temporary storage location.
- A return at the end of ISR reloads the PC from that temporary storage location.
- This causes the execution to resume at instruction i+1.
- When processor is handling interrupts, it must inform device that its request has been recognized.
- This may be accomplished by INTA signal.
- The task of saving and restoring the information can be done automatically by the processor.
- The processor saves only the contents of **PC & Status register.**
- Saving registers also increases the Interrupt Latency.
- **Interrupt Latency** is a delay between
  → time an interrupt-request is received and
  → start of the execution of the ISR.
- Generally, the long interrupt latency in unacceptable.

Difference between Subroutine & ISR

Subroutine	ISR
A subroutine performs a function required by the program from which it is called.	ISR may not have anything in common with program being executed at time INTR is received
Subroutine is just a linkage of 2 or more function related to each other.	Interrupt is a mechanism for coordinating I/O transfers.

INTERRUPT HARDWARE

- Most computers have several I/O devices that can request an interrupt.
- A single interrupt-request (IR) line may be used to serve n devices (Figure 4.6).
- All devices are connected to IR line via switches to ground.
- To request an interrupt, a device closes its associated switch.
- Thus, if all IR signals are inactive, the voltage on the IR line will be equal to V_{dd}.
- When a device requests an interrupt, the voltage on the line drops to 0.
- This causes the INTR received by the processor to go to 1.
- The value of INTR is the logical OR of the requests from individual devices.

  $INTR = INTR_1 + INTR_{2+} \ldots\ldots + INTR_n$
- A special gates known as open-collector or open-drain are used to drive the INTR line.
- The Output of the open collector control is equal to a switch to the ground that is
  - → open when gates input is in "0" state and
  - → closed when the gates input is in "1" state.
- Resistor R is called a **Pull-up Resistor** because
  it pulls the line voltage up to the high-voltage state when the switches are open.

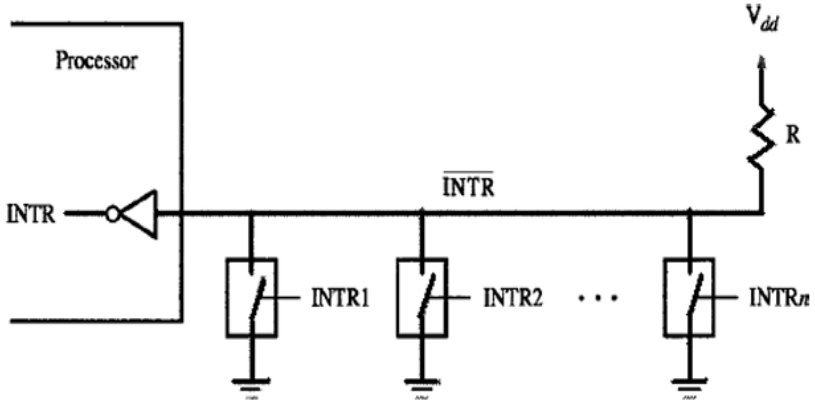


Figure 4.6 An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.

ENABLING & DISABLING INTERRUPTS

- All computers fundamentally should be able to enable and disable interruptions as desired.
- The problem of infinite loop occurs due to successive interruptions of active INTR signals.
- There are 3 mechanisms to solve problem of infinite loop:
  1) Processor should ignore the interrupts until execution of first instruction of the ISR.
  2) Processor should automatically disable interrupts before starting the execution of the ISR.
  3) Processor has a special INTR line for which the interrupt-handling circuit.
     Interrupt-circuit responds only to leading edge of signal. Such line is called edge-triggered.
- Sequence of events involved in handling an interrupt-request:
  1) The device raises an interrupt-request.
  2) The processor interrupts the program currently being executed.
  3) Interrupts are disabled by changing the control bits in the processor status register (PS).
  4) The device is informed that its request has been recognized.
     In response, the device deactivates the interrupt-request signal.
  5) The action requested by the interrupt is performed by the interrupt-service routine.
  6) Interrupts are enabled and execution of the interrupted program is resumed.

POLLING

- Information needed to determine whether device is requesting interrupt is available in status-regis
- Following condition-codes are used:
  - DIRQ → Interrupt-request for display.
  - KIRQ → Interrupt-request for keyboard.
  - KEN → keyboard enable.
  - DEN → Display Enable.
  - SIN, SOUT → status flags.
- For an input device, SIN status flag in used.

  SIN = 1 → when a character is entered at the keyboard.

  SIN = 0 → when the character is read by processor.

  IRQ=1 → when a device raises an interrupt-requests (Figure 4.3).
- Simplest way to identify interrupting-device is to have ISR poll all devices connected to bus.
- The first device encountered with its IRQ bit set is serviced.
- After servicing first device, next requests may be serviced.
- **Advantage:** Simple & easy to implement.

  Disadvantage: More time spent polling IRQ bits of all devices.

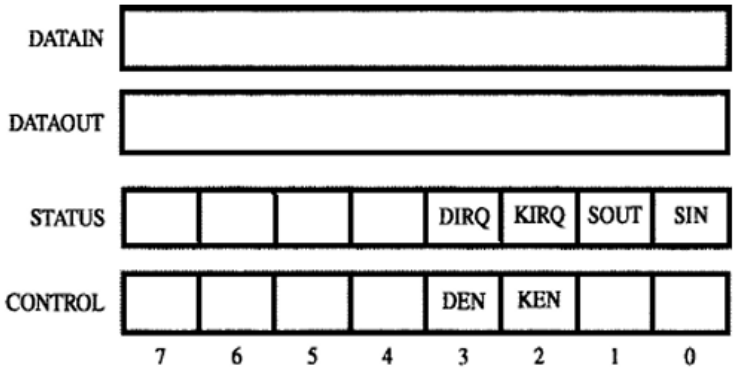


Figure 4.3 Registers in keyboard and display interfaces.

```
        Move      #LINE,R0          Initialize memory pointer.
WAITK   TestBit   #0,STATUS         Test SIN.
        Branch=0  WAITK             Wait for character to be entered.
        Move      DATAIN,R1         Read character.
WAITD   TestBit   #1,STATUS         Test SOUT.
        Branch=0  WAITD             Wait for display to become ready.
        Move      R1,DATAOUT        Send character to display.
        Move      R1,(R0)+          Store charater and advance pointer.
        Compare   #$0D,R1           Check if Carriage Return.
        Branch≠0  WAITK             If not, get another character.
        Move      #$0A,DATAOUT      Otherwise, send Line Feed.
        Call      PROCESS           Call a subroutine to process the
                                      the input line.
```

Figure 4.4 A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display.

DIRECT MEMORY ACCESS (DMA)

• The transfer of a block of data directly b/w an external device & main-memory w/o continuous involvement by processor is called DMA.

• DMA controller
  → is a control circuit that performs DMA transfers (Figure 8.13).
  → is a part of the I/O device interface.
  → performs the functions that would normally be carried out by processor.

• While a DMA transfer is taking place, the processor can be used to execute another program.

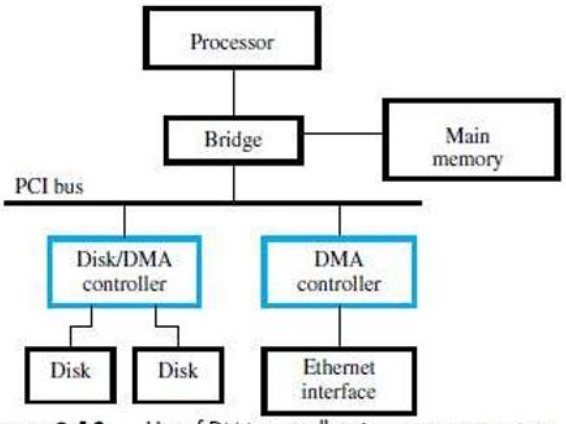


Figure 8.13 Use of DMA controllers in a computer system.

• DMA interface has three registers (Figure 8.12):
  1) First register is used for storing starting-address.
  2) Second register is used for storing word-count.
  3) Third register contains status- & control-flags.

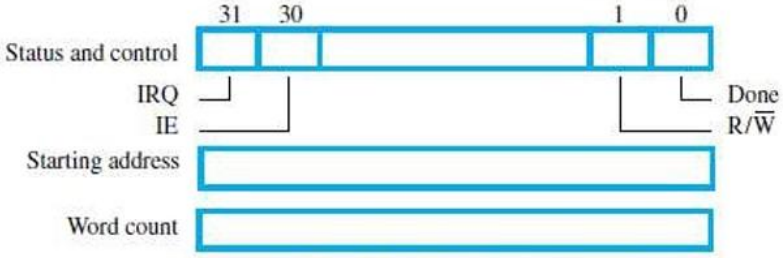


Figure 8.12 Typical registers in a DMA controller.

• The R/W bit determines direction of transfer.
  If R/W=1, controller performs a read-operation (i.e. it transfers data from memory to I/O),
  Otherwise, controller performs a write-operation (i.e. it transfers data from I/O to memory).

• If Done=1, the controller
  → has completed transferring a block of data and
  → is ready to receive another command. (IE → Interrupt Enable).

• If IE=1, controller raises an interrupt after it has completed transferring a block of data.

• If IRQ=1, controller requests an interrupt.

• Requests by DMA devices for using the bus are always given higher priority than processor requests.

• There are 2 ways in which the DMA operation can be carried out:
  1) Processor originates most memory-access cycles.
  ➢ DMA controller is said to "steal" memory cycles from processor.
  ➢ Hence, this technique is usually called **Cycle Stealing**.
  2) DMA controller is given exclusive access to main-memory to transfer a block of data without any interruption. This is known as **Block Mode** (or burst mode).

BUS ARBITRATION

- The device that is allowed to initiate data-transfers on bus at any given time is called **bus-master**.
- There can be only one bus-master at any given time.
- **Bus Arbitration** is the process by which
  - → next device to become the bus-master is selected &
  - → bus-mastership is transferred to that device.
- The two approaches are:
  - **1) Centralized Arbitration:** A single bus-arbiter performs the required arbitration.
  - **2) Distributed Arbitration:** All devices participate in selection of next bus-master.
- A conflict may arise if both the processor and a DMA controller or two DMA controllers try to use the bus at the same time to access the main-memory.
- To resolve this, an arbitration procedure is implemented on the bus to coordinate the activities of all devices requesting memory transfers.
- The bus arbiter may be the processor or a separate unit connected to the bus.

CENTRALIZED ARBITRATION

- A single bus-arbiter performs the required arbitration (Figure: 4.20).
- Normally, processor is the bus-master.
- Processor may grant bus-mastership to one of the DMA controllers.
- A DMA controller indicates that it needs to become bus-master by activating BR line.
- The signal on the BR line is the logical OR of bus-requests from all devices connected to it.
- Then, processor activates BG1 signal indicating to DMA controllers to use bus when it becomes free.
- BG1 signal is connected to all DMA controllers using a daisy-chain arrangement.
- If DMA controller-1 is requesting the bus,
  Then, DMA controller-1 blocks propagation of grant-signal to other devices.
  Otherwise, DMA controller-1 passes the grant downstream by asserting BG2.
- Current bus-master indicates to all devices that it is using bus by activating BBSY line.
- The bus-arbiter is used to coordinate the activities of all devices requesting memory transfers.
- Arbiter ensures that only 1 request is granted at any given time according to a priority scheme.
  (BR → Bus-Request, BG → Bus-Grant, BBSY → Bus Busy).

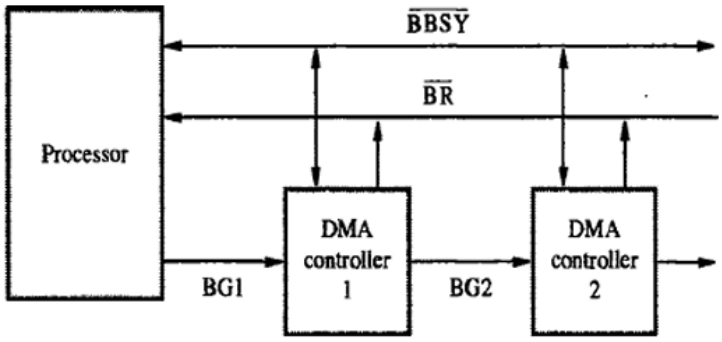


Figure 4.20 A simple arrangement for bus arbitration using a daisy chain.

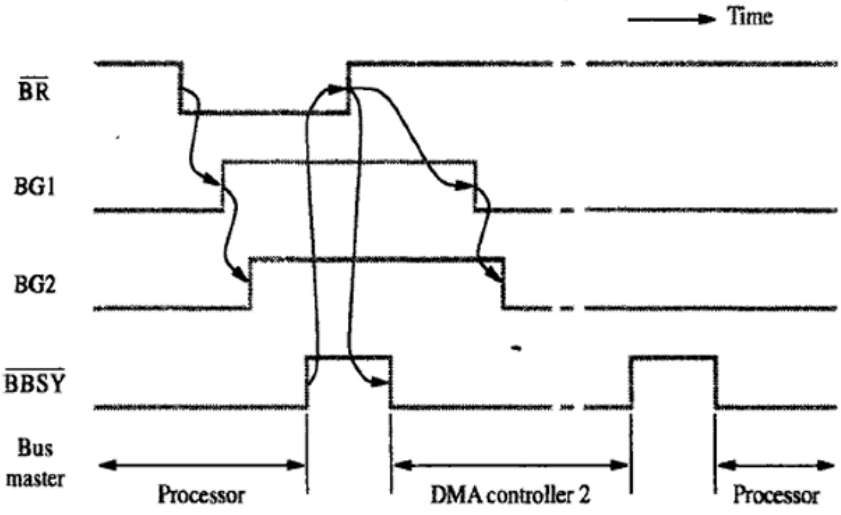


Figure 4.21 Sequence of signals during transfer of bus mastership for the devices in Figure 4.20.

- The timing diagram shows the sequence of events for the devices connected to the processor.
- DMA controller-2
  → requests and acquires bus-mastership and
  → later releases the bus. (Figure: 4.21).
- After DMA controller-2 releases the bus, the processor resources bus-mastership.

DISTRIBUTED ARBITRATION

- All device participate in the selection of next bus-master (Figure 4.22).
- Each device on bus is assigned a 4-bit identification number (ID).
- When 1 or more devices request bus, they
  → assert Start-Arbitration signal &
  → place their 4-bit ID numbers on four open-collector lines $\overline{ARB\,0}$ through $ARB\,3$.
- A winner is selected as a result of interaction among signals transmitted over these lines.
- Net-outcome is that the code on 4 lines represents request that has the highest ID number.
- **Advantage:**
  This approach offers higher reliability since operation of bus is not dependent on any single device.

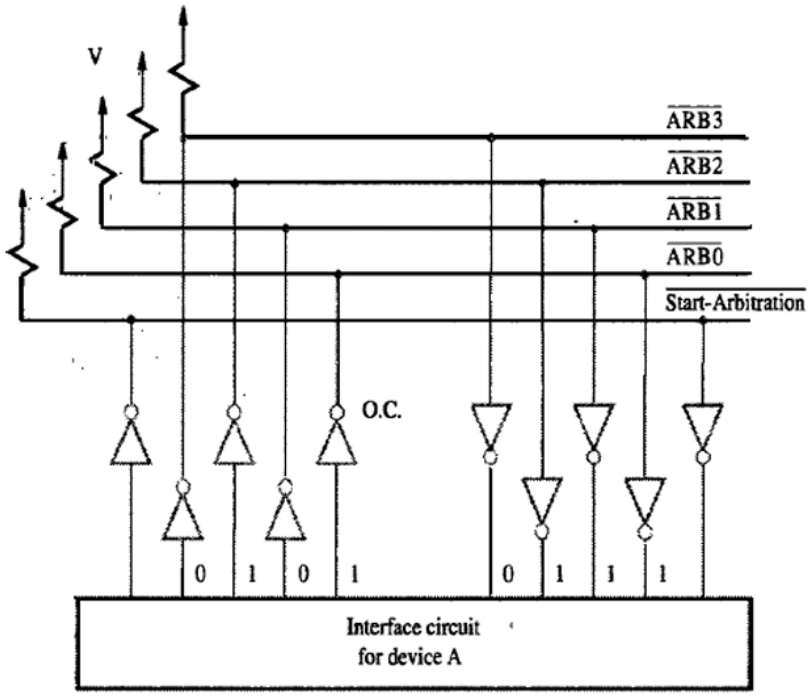


Figure 4.22 A distributed arbitration scheme.

For example:

- Assume 2 devices A & B have their ID 5 (0101), 6 (0110) and their code is 0111.
- Each device compares the pattern on the arbitration line to its own ID starting from MSB.
- If the device detects a difference at any bit position, it disables the drivers at that bit position.
- Driver is disabled by placing "0" at the input of the driver.
- In e.g. "A" detects a difference in line ARB1, hence it disables the drivers on lines ARB1 & ARB0.
- This causes pattern on arbitration-line to change to 0110. This means that "B" has won contention.

BUS

- Bus → is used to inter-connect main-memory, processor & I/O-devices
  → includes lines needed to support interrupts & arbitration.

- Primary function: To provide a communication-path for transfer of data.
- **Bus protocol** is set of rules that govern the behavior of various devices connected to the buses.
- Bus-protocol specifies parameters such as:
  → asserting control-signals
  → timing of placing information on bus
  → rate of data-transfer.
- A typical bus consists of 3 sets of lines:
  1) Address,
  2) Data &
  3) Control lines.
- Control-signals
  → specify whether a read or a write-operation is to be performed.
  → carry timing information i.e. they specify time at which I/O-devices place data on the bus.
- R/W line specifies
  → read-operation when R/W=1.
  → write-operation when R/W=0.
- During data-transfer operation,
  - One device plays the role of a bus-master.
  - Master-device initiates the data-transfer by issuing read/write command on the bus.
  - The device addressed by the master is called as Slave.
- Two types of Buses: 1) Synchronous and 2) Asynchronous.

SYNCHRONOUS BUS

- All devices derive timing-information from a common clock-line.
- Equally spaced pulses on this line define equal time intervals.
- During a "bus cycle", one data-transfer can take place.

A sequence of events during a read-operation

- At time t_0, the master (processor)
  - → places the device-address on address-lines &
  - → sends an appropriate command on control-lines (Figure 7.3).
- The command will
  - → indicate an input operation &
  - → specify the length of the operand to be read.
- Information travels over bus at a speed determined by physical & electrical characteristics.
- Clock pulse width(t_1-t_0) must be longer than max. propagation-delay b/w devices connected to bus.
- The clock pulse width should be long to allow the devices to decode the address & control signals.
- The slaves take no action or place any data on the bus before t_1.
- Information on bus is unreliable during the period t_0 to t_1 because signals are changing state.
- Slave places requested input-data on data-lines at time t_1.
- At end of clock cycle (at time t_2), master strobes (captures) data on data-lines into its input-buffer
- For data to be loaded correctly into a storage device,
  data must be available at input of that device for a period greater than setup-time of device.

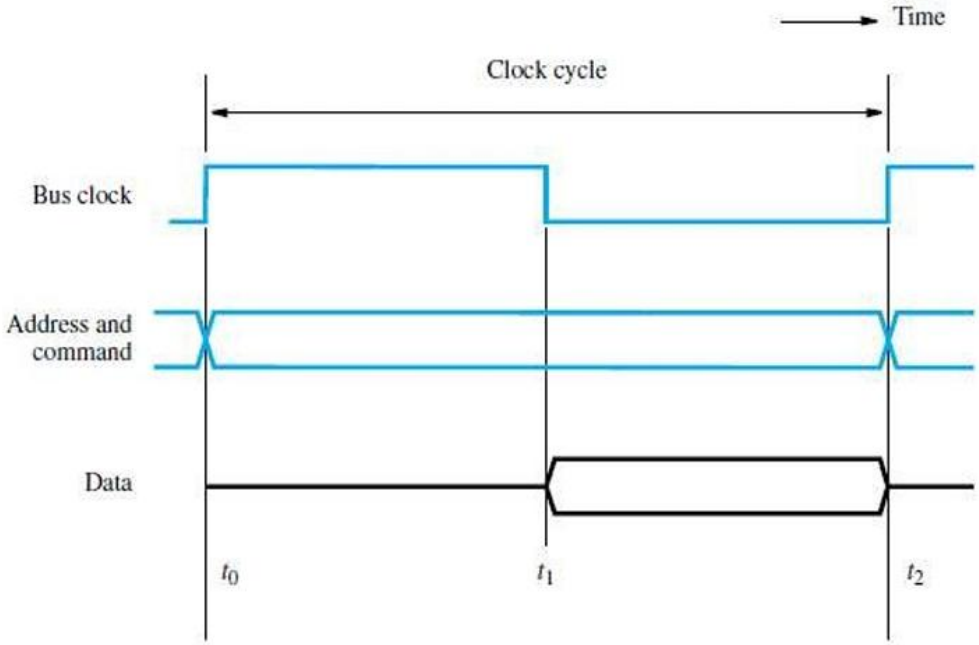


Figure 7.3 Timing of an input transfer on a synchronous bus.

A Detailed Timing Diagram for the Read-operation

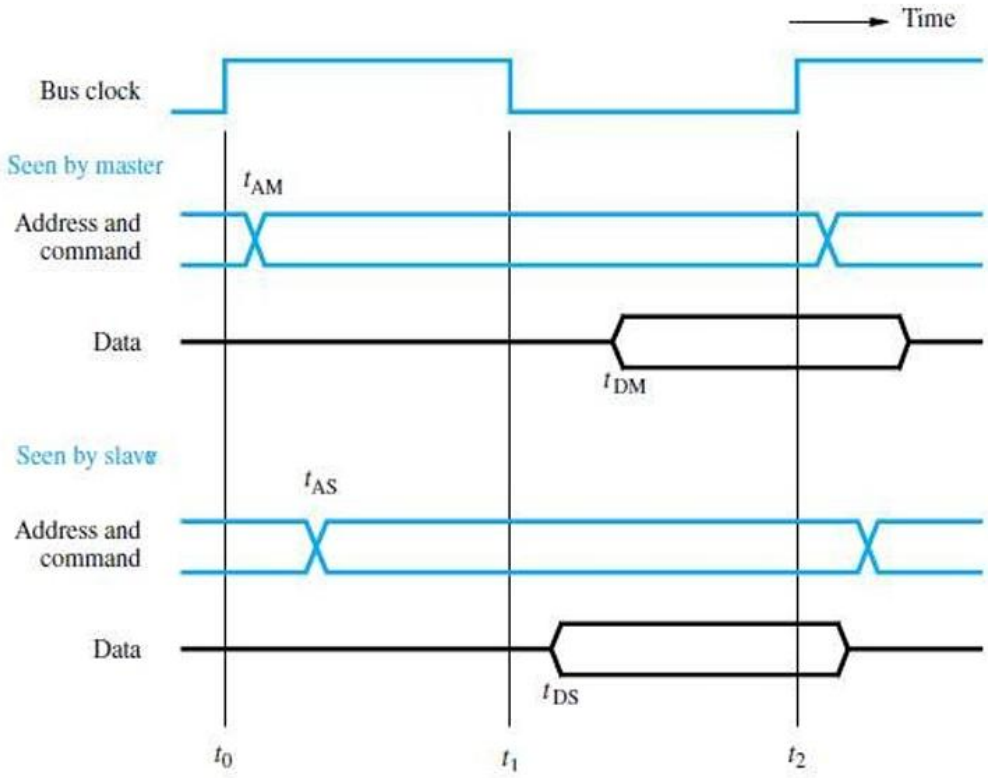


Figure 7.4 A detailed timing diagram for the input transfer of Figure 7.3.

- The picture shows two views of the signal except the clock (Figure 7.4).
- One view shows the signal seen by the master & the other is seen by the salve.
- Master sends the address & command signals on the rising edge at the beginning of clock period (t_0).
- These signals do not actually appear on the bus until t_{am}.
- Sometimes later, at t_{AS} the signals reach the slave.
- The slave decodes the address.
- At t_1, the slave sends the requested-data.
- At t_2, the master loads the data into its input-buffer.
- Hence the period t_2, t_{DM} is the setup time for the master"s input-buffer.
- The data must be continued to be valid after t_2, for a period equal to the hold time of that buffers.

Disadvantages

- The device does not respond.
- The error will not be detected.

Multiple Cycle Transfer for Read-operation

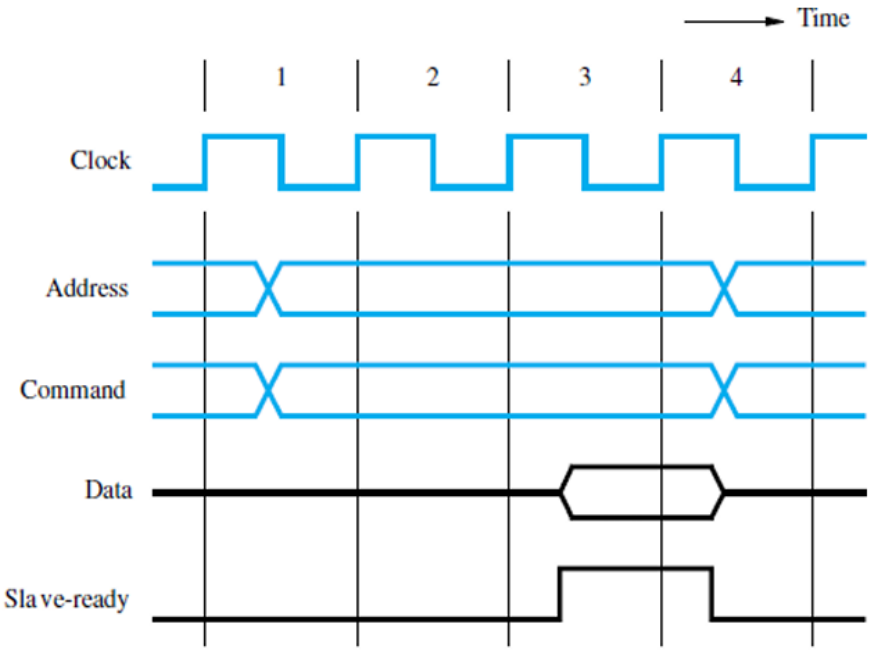


Figure 7.5 An input transfer using multiple clock cycles.

- During, clock cycle-1, master sends address/command info the bus requesting a "read" operation.
- The slave receives & decodes address/command information (Figure 7.5).
- At the active edge of the clock i.e. the beginning of clock cycle-2, it makes accession to respond immediately.
- The data become ready & are placed in the bus at clock cycle-3.
- At the same times, the slave asserts a control signal called **slave-ready**.
- The master strobes the data to its input-buffer at the end of clock cycle-3.
- The bus transfer operation is now complete.
- And the master sends a new address to start a new transfer in clock cycle4.
- The slave-ready signal is an acknowledgement from the slave to the master.

ASYNCHRONOUS BUS

- This method uses handshake-signals between master and slave for coordinating data-transfers.
- There are 2 control-lines:
  - **1) Master-Ready (MR)** is used to indicate that master is ready for a transaction.
  - **2) Slave-Ready (SR)** is used to indicate that slave is ready for a transaction.

The Read Operation proceeds as follows:

- At t_0, master places address/command information on bus.
- At t_1, master sets MR-signal to 1 to inform all devices that the address/command-info is ready.
  - MR-signal =1 → causes all devices on the bus to decode the address.
  - The delay $t_1 - t_0$ is intended to allow for any skew that may occurs on the bus.
  - Skew occurs when 2 signals transmitted from 1 source arrive at destination at different time
  - Therefore, the delay $t_1 - t_0$ should be larger than the maximum possible bus skew.
- At t_2, slave
  - → performs required input-operation &
  - → sets SR signal to 1 to inform all devices that it is ready (Figure 7.6).
- At t_3, SR signal arrives at master indicating that the input-data are available on bus.
- At t_4, master removes address/command information from bus.
- At t_5, when the device-interface receives the 1-to-0 transition of MR signal, it removes data and SR signal from the bus. This completes the input transfer.

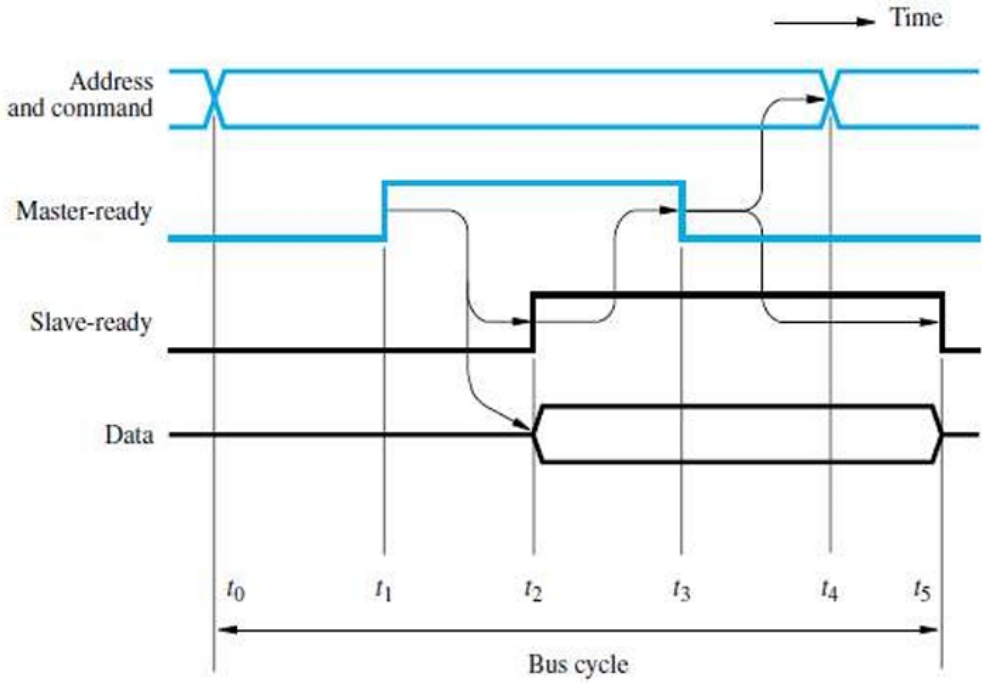


Figure 7.6 Handshake control of data transfer during an input operation.

- A change of state is one signal is followed by a change is the other signal. Hence this scheme is called as **Full Handshake**.
- **Advantage:** It provides the higher degree of flexibility and reliability.

INTERFACE-CIRCUITS

- An **I/O Interface** consists of the circuitry required to connect an I/O device to a computer-bus.
- On one side of the interface, we have bus signals.
  On the other side, we have a data path with its associated controls to transfer data between the interface and the I/O device known as **port**.
- Two types are:
  1. Parallel Port transfers data in the form of a number of bits (8 or 16) simultaneously to or from the device.
  2. Serial Port transmits and receives data one bit at a time.
- Communication with the bus is the same for both formats.
- The conversion from the parallel to the serial format, and vice versa, takes place inside the interface-circuit.
- In parallel-port, the connection between the device and the computer uses
  → a multiple-pin connector and
  → a cable with as many wires.
- This arrangement is suitable for devices that are physically close to the computer.
- In serial port, it is much more convenient and cost-effective where longer cables are needed.

Functions of I/O Interface

1) Provides a storage buffer for at least one word of data.
2) Contains status-flags that can be accessed by the processor to determine whether the buffer is full or empty.
3) Contains address-decoding circuitry to determine when it is being addressed by the processor.
4) Generates the appropriate timing signals required by the bus control scheme.
5) Performs any format conversion that may be necessary to transfer data between the bus and the I/O device (such as parallel-serial conversion in the case of a serial port).

PARALLEL-PORT
KEYBOARD INTERFACED TO PROCESSOR

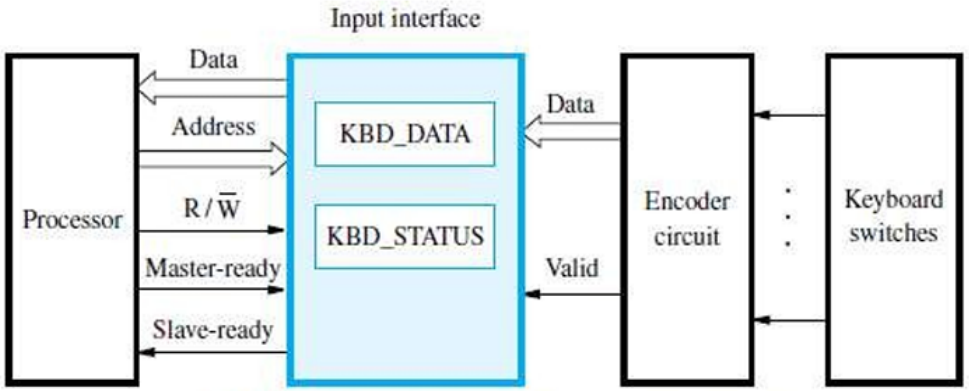


Figure 7.10 Keyboard to processor connection.

- The output of the encoder consists of
  - → bits representing the encoded character and
  - → one signal called **valid**, which indicates the key is pressed.
- The information is sent to the interface-circuits (Figure 7.10).
- Interface-circuits contain
  1) Data register DATAIN &
  2) Status-flag SIN.
- When a key is pressed, the Valid signal changes from 0 to1.
  Then, SIN=1 → when ASCII code is loaded into DATAIN.
  SIN = 0 → when processor reads the contents of the DATAIN.
- The interface-circuit is connected to the asynchronous bus.
- Data transfers on the bus are controlled using the handshake signals:
  1) Master ready &
  2) Slave ready.

INPUT-INTERFACE-CIRCUIT

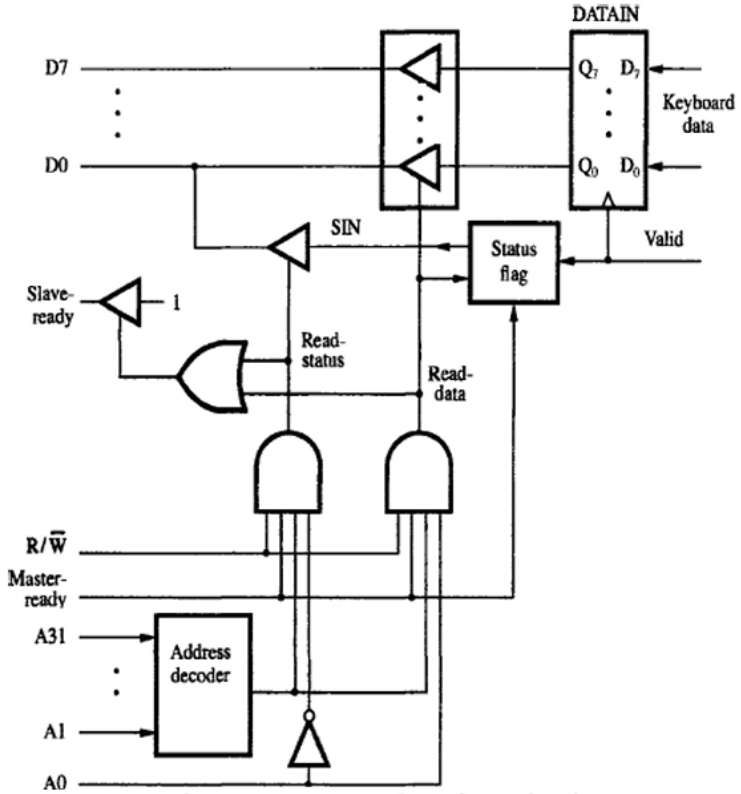


Figure 4.29: Input-interface-circuit

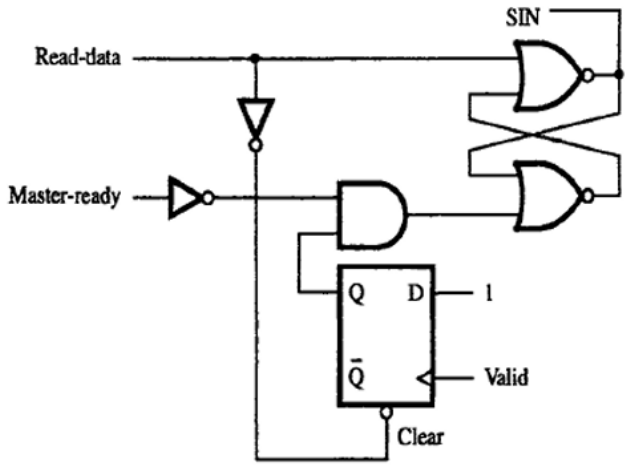


Figure 4.30 Circuit for the status flag block in Figure 4.29.

- Output-lines of DATAIN are connected to the data-lines of bus by means of 3-state drivers (Fig 4.29).
- Drivers are turned on when
  - → processor issues a read signal and
  - → address selects DATAIN.
- SIN signal is generated using a status-flag circuit (Figure 4.30).
  SIN signal is connected to line D_0 of the processor-bus using a 3-state driver.
- Address-decoder selects the input-interface based on bits A_1 through A_{31}.
- Bit A_0 determines whether the status or data register is to be read, when Master-ready is active.
- Processor activates the Slave-ready signal, when either the Read-status or Read-data is equal to 1.

PRINTER INTERFACED TO PROCESSOR

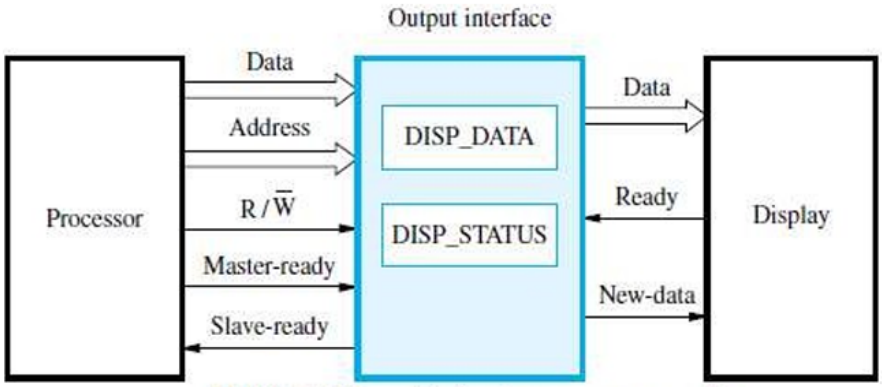


Figure 7.13 Display to processor connection.

- Keyboard is connected to a processor using a parallel-port.
- Processor uses
  - → memory-mapped I/O and
  - → asynchronous bus protocol.
- On the processor-side of the interface, we have:
  - → Data-lines
  - → Address-lines
  - → Control or R/W line
  - → Master-Ready signal and
  - → Slave-Ready signal.
- On the keyboard-side of the interface, we have:
  - → Encoder-circuit which generates a code for the key pressed.
  - → Debouncing-circuit which eliminates the effect of a key.
  - → Data-lines which contain the code for the key.
  - → Valid line changes from 0 to 1 when the key is pressed. This causes the code to be loaded into DATAIN and SIN to be set to 1.

GENERAL 8 BIT PARALLEL PROCESSING

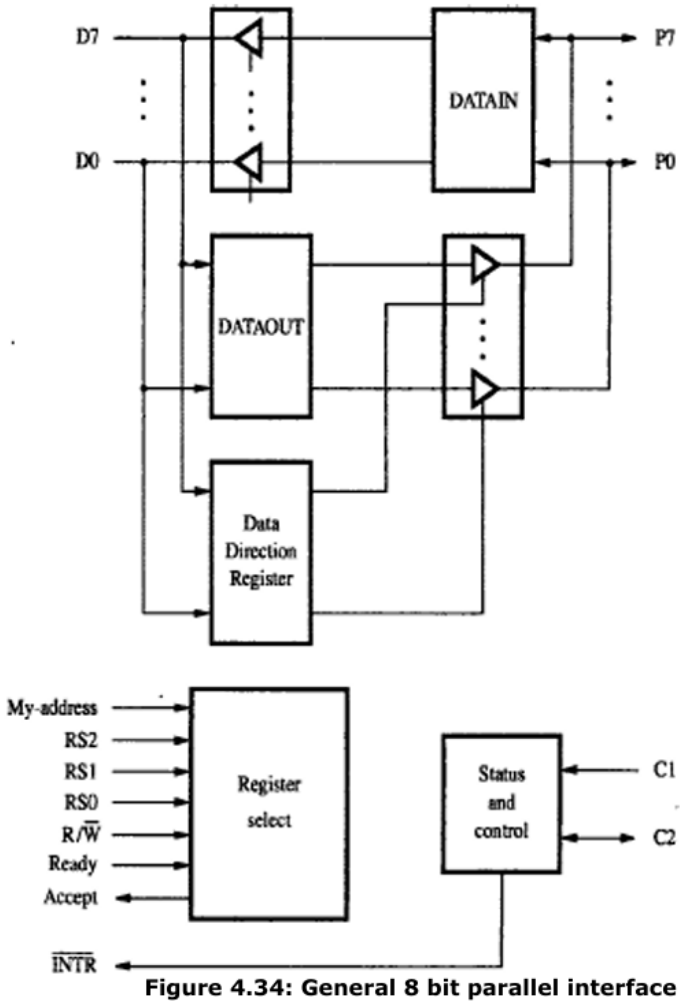


Figure 4.34: General 8 bit parallel interface

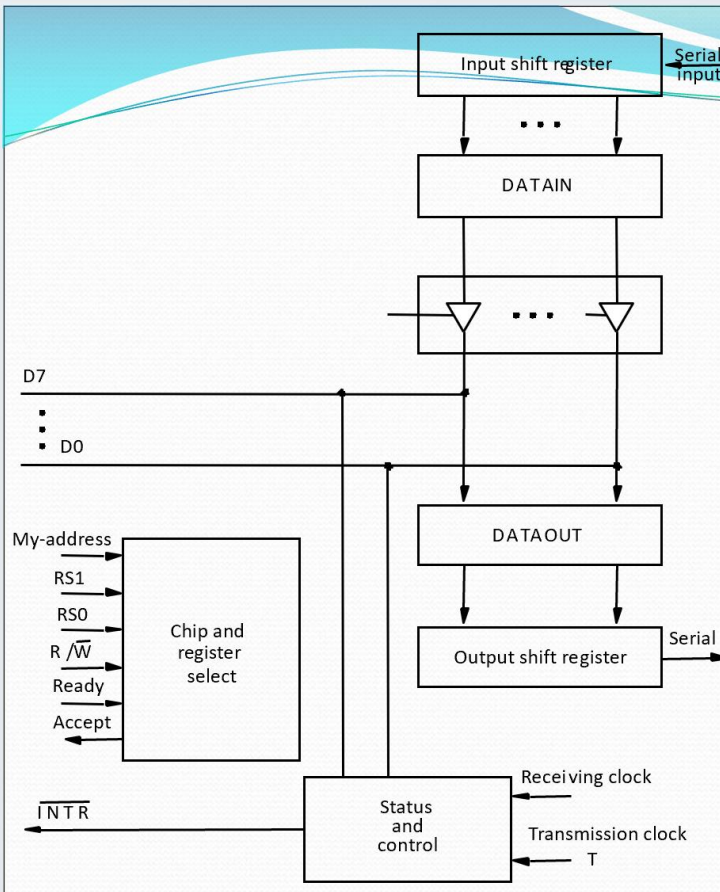
- Data-lines $\mathbf{P_7}$ through $\mathbf{P_o}$ can be used for either input or output purposes (Figure 4.34).
- For increased flexibility,
  → some lines can be used as inputs and
  → some lines can be used as outputs.
- The **DATAOUT** register is connected to data-lines via 3-state drivers that are controlled by a **DDR**.
- The processor can write any 8-bit pattern into DDR. (DDR → Data Direction Register).
- If DDR=1,
  Then, data-line acts as an output-line;
  Otherwise, data-line acts as an input-line.
- Two lines, $\mathbf{C_1}$ and $\mathbf{C_2}$ are used to control the interaction between interface-circuit and I/0 device.
  Two lines, C_1 and C_2 are also programmable.
- Line C_2 is bidirectional to provide different modes of signaling, including the handshake.
- The **Ready** and **Accept** lines are the handshake control lines on the processor-bus side.
  Hence, the Ready and Accept lines can be connected to Master-ready and Slave-ready.
- The input signal **My-address** should be connected to the output of an address-decoder.
  The address-decoder recognizes the address assigned to the interface.
- There are 3 register select lines: $\mathbf{RS_0}$-$\mathbf{RS_2}$.
  Three register select lines allows up to eight registers in the interface.
- An interrupt-request **INTR** is also provided.
  INTR should be connected to the interrupt-request line on the computer-bus.

Functions of I/O interface

- Provides a **storage buffer** for at least of one word of data.
- Contains **status flags** that can be accessed by processor to determine whether the buffer is full(for input) or empty(for output).
- Contains **address decoding** circuitry to determine when it is being addressed by the processor.
- Generates the appropriate **timing signals** required by the bus control scheme.
- Performs any **format conversion** that may be necessary to transfer data between the bus and I/O device, such as Parallel-serial conversion in the case of a serial port.

Serial port

- Serial port is used to connect the processor to I/O devices that require transmission of data one bit at a time.
- Serial port communicates in a bit-serial fashion on the device side and bit parallel fashion on the bus side.
  - Transformation between the parallel and serial formats is achieved with shift registers that have parallel access capability.



- *Input shift register accepts input one bit at a time from the I/O device.*
- *Once all the 8 bits are received, the contents of the input shift register are loaded in parallel into DATAIN register.*
- *Output data in the DATAOUT register are loaded into the output shift register.*
- *Bits are shifted out of the output shift register and sent out to the I/O device one bit at a time.*
- *As soon as data from the input shift reg. are loaded into DATAIN, it can start accepting another 8 bits of data.*
- *Input shift register and DATAIN registers are both used at input so that the input shift register can start receiving another set of 8 bits from the input device after loading the contents to DATAIN, before the processor reads the contents of DATAIN. This is called as double-buffering.*